

(Task 1 of 20) In this tutorial, we will learn *more* about **mutable values**, illustrated with **vectors**.

0

(Task 2 of 20) What is the result of running this program?

Lispy |

Lispy [Run ▶]	JavaScript
(defvar x (mvec 19 73 28))	let x = [19, 73, 28];
(defvar y (mvec x x))	let y = [x, x];
(vec-set! (vec-ref y 0) 0 64)	y[0][0] = 64;
(vec-ref y 1)	console.log(y[1]);

#(64 73 28)

2

You predicted the output correctly 🎉🎉🎉

3

This program first creates a three-element vector and binds it to `x`. After that, the program creates a two-element vector. Both elements refer to the first vector. After that, the 0-th element of the three-element vector is replaced with 64. Finally, the three-element vector is printed.

Click [here](#) to run this program in the Stacker.

(Task 3 of 20) What is the result of running this program?

Lispy |

Lispy [Run ▶]	Pseudo
(defvar x (mvec 43 54))	let x = vec[43, 54]
(vec-set! x 0 x)	x[0] = x
(vec-ref x 1)	print(x[1])

54

5

Please briefly explain why you think the answer is 54.

6

vec-ref x 1 is unchanged by whatever the recursive set of element 0 does

7

You predicted the output correctly 🎉🎉🎉

8

`x` is bound to a vector. `(vec-set! x 0 x)` makes the vector refer to itself. This is fine because a vector element can be any value, including itself. Besides, the vector is not copied, so the mutation finishes immediately.

Click [here](#) to run this program in the Stacker

Click [here](#) to run this program in the Stacker.

Lispy | 

(Task 4 of 20) What is the result of running this program?

Lispy [Run 

Scala 3

```
(defvar x (mvec 74 82))  val x = Buffer[Any](74, 82)
(defvar y (mvec x))      val y = Buffer(x)
(vec-set! x 0 y)         x(0) = y
(vec-ref x 1)            println(x(1))
```

82

10

You predicted the output correctly 🎉🎉🎉

11

The program binds `x` to a two-element vector and `y` to a one-element vector. After that, it replaces the `0`-th element of the two-element vector with the one-element vector. The other element of the two-element vector is still `82`.

Click [here](#) to run this program in the Stacker.

Lispy | 

(Task 5 of 20) What is the result of running this program?

Lispy [Run 

JavaScript

```
(defvar x (mvec 52 70 61))  let x = [ 52, 70, 61 ];
(defvar y x)               let y = x;
(vec-set! x 0 93)          x[0] = 93;
(vec-ref y 0)              console.log(y[0]);
```

93

13

You predicted the output correctly 🎉🎉🎉

14

The program creates a vector where the `0`-th element is `52`, and binds it to both `x` and `y`. The `0`-th element of the vector is then replaced with `93`. So, the result is `93`.

Click [here](#) to run this program in the Stacker.

Lispy | 

(Task 6 of 20) What is the result of running this program?

Lispy [Run 

Scala 3

```
(defvar x (mvec 99 83))  val x = Buffer(99, 83)
(deffun (f y)           def f(y : Buffer[Int]) =
  (vec-set! x 0 34)      x(0) = 34
  y)                    y
  (f x))                (f(x))
```

<code>(f x)</code>	<code>println(f(x))</code>	
<code>#(34 83)</code>		16
You predicted the output correctly 🎉🎉🎉 The program creates a vector and binds it to <code>x</code>. The function call <code>(f x)</code> binds <code>y</code> to the same vector. The function replaces the 0-th element of the vector with 34 and then returns the vector. So, the final result is the vector, which is printed as <code>#(34 83)</code>. Click here to run this program in the Stacker.		17

(Task 7 of 20) What is the result of running this program?		Lispy 
Lispy [Run ▶] <pre>(defvar x (mvec 62)) (defvar y x) (vec-set! y 0 34) x</pre>	JavaScript <pre>let x = [62]; let y = x; y[0] = 34; console.log(x);</pre>	
<code>#(34)</code>		19
You predicted the output correctly 🎉🎉🎉 <code>x</code> is first bound to a vector that refers 62. <code>y</code> is bound to the same vector. The vector mutation replaces 62 with 34. So, eventually, the vector becomes <code>#(34)</code>. Click here to run this program in the Stacker.		20

(Task 8 of 20) What is the result of running this program?		Lispy 
Lispy [Run ▶] <pre>(defvar x (mvec 10 48 95)) (deffun (f y) (vec-set! y 0 32) (vec-ref y 0)) (defvar z (f x)) (vec-ref x 0)</pre>	JavaScript <pre>let x = [10, 48, 95]; function f(y) { y[0] = 32; return y[0]; } let z = f(x); console.log(x[0]);</pre>	
<code>32</code>		22
You predicted the output correctly 🎉🎉🎉		23

The program first creates a three-element vector, where the 0-th element is 10. After that, the program defines a function `f`. The function call `(f x)` replaces the 0-th vector element with 32. After that, the vector is printed. So the result is 32.

Click [here](#) to run this program in the Stacker.

(Task 9 of 20) What is the result of running this program?

Lispy | 

Lispy [Run]

Scala 3

```
(defvar x (mvec 53))    val x = Buffer(53)
(defvar v (mvec 72 x))  val v = Buffer(72, x)
(vec-set! x 0 72)       x(0) = 72
v                       println(v)
```

`#(72 #(72))`

25

You predicted the output correctly 🎉🎉🎉

26

`x` is bound to a one-element vector. The only element of this vector is 53. `v` is bound to a two-element vector. The elements of this vector are 72 and the one-element vector. The subsequent `(vec-set! x 0 72)` mutates the one-element vector. Finally, the value of `v` is printed.

Click [here](#) to run this program in the Stacker.

(Task 10 of 20) What did you learn about vectors from these programs?

27

Mutable and passed by reference

28

(Task 11 of 20) A vector can be referred to by more than one variable and even by other vectors (including itself). Referring to a vector does not create a copy of the vector; rather, they share the same vector. Specifically

29

- Binding a vector to a new variable does not create a copy of that vector.
- Vectors that are passed to a function in a function call do not get copied.
- Creating new vectors that refer to existing ones does not create new copies of the existing vectors.

The references share the same vector. That is, vectors can be **aliased**.

Any feedback regarding these statements? Feel free to skip this question.

30

Not obvious they would work this way

31

Thank you!

32

(Task 12 of 20) Now please scroll back and select 1-3 programs that make the above points.

33

You don't need to select *all* such programs.

(You selected 1 programs)

34

Okay. How does this program (24) support the point?

35

shows

36

(Task 13 of 20) Reconsider these two programs that you might have seen. The only difference is in (defvar y ____).

Lispy | 

Lispy 	JavaScript
(defvar x (mvec 52 96))	let x = [52, 96];
(defvar y x)	let y = x;
(vec-set! x 0 34)	x[0] = 34;
y	console.log(y);

Lispy 	JavaScript
(defvar x (mvec 52 96))	let x = [52, 96];
(defvar y (mvec 52 96))	let y = [52, 96];
(vec-set! x 0 34)	x[0] = 34;
y	console.log(y);

It is tempting to describe the variables as

- x is bound to #(52 96)
- y is bound to #(52 96)

This description does not help us to understand the program because it can't explain why (vec-set! x 0 34) mutates y in one case, and does not in the other.

What is a better way to describe the bindings that help solve this problem?

y is bound to x

38

We can sav, in the first program

39

- `x` is bound to `@100`
- `y` is bound to `@100`

where

- `@100` is `#(52 96)`

While for the other program

- `x` is bound to `@100`
- `y` is bound to `@200`

where

- `@100` is `#(52 96)`
- `@200` is `#(52 96)`

(Task 14 of 20) In SMoL, each vector has its own unique **heap address** (e.g., `@100` and `@200`). The mapping from addresses to vectors is called the **heap**.

40

(Note: we use `@ddd` (e.g., `@123`, `@200`, and `@100`) to represent heap addresses. Heap addresses are *random*. The numbers don't mean anything.)

Any feedback regarding these statements? Feel free to skip this question.

41

helpful

42

Thank you!

43

(Task 15 of 20) Which choice best describes the status of the heap at the end of the following program?

Lispy | Lispy [Run 

Pseudo

```
(defvar x 3)      let x = 3
(defvar v (mvec 1 2 x))  let v = vec[1, 2, x]
```

45

- A. `@1 = #(1 2 3)`
- B. `@1 = #(1 2 x)`
- C. There is nothing in the heap.

@1 = #(1 2 3)

46

You gave the correct answer 🎉🎉🎉

47

C is wrong because the `(mvec 1 2 x)` creates a vector. Every vector is stored on the heap.

B is wrong because vectors refer values, while `x` is not a value.

(Task 16 of 20) Which choice best describes the status of the heap at the end of the following program?

Lispy | 

Lispy [Run]

Scala 3

```
(defvar mv (mvec 3))      val mv = Buffer(3)
(defvar mv2 (mvec mv mv)) val mv2 = Buffer(mv, mv)
(vec-set! (vec-ref mv2 0) 0 42) mv2(0)(0) = 42
```

49

- A. @100 = #(3); @200 = #(@100 @100)
- B. @100 = #(3); @200 = #(@300 @100); @300 = #(42)
- C. @100 = #(3); @200 = #(#(42) #(3))
- D. @100 = #(42); @200 = #(@100 @100)
- E. @100 = #(42); @200 = #(#(42) #(42))
- F. There is nothing in the heap.

@100 = #(42); @200 = #(@100 @100)

50

You gave the correct answer 🎉🎉🎉

51

E and **C** are wrong. Vectors refer values. `#(42)` and `#(3)` are not values, although some vector values are printed like them. So, `@200 = #(#(42) #(42))` and `@200 = #(#(42) #(3))` can not be valid.

`(mvec 3)` creates a 1-element vector, `@100`. The only element is 3. `(mvec mv mv)` creates a 2-element vector, `@200`. Both elements of `@200` are `@100`. No more vectors are created, which means **B** must be wrong. So, then, the correct answer must be **A** or **D**.

However, the subsequent mutation changes `@100` (the first element of `@200`). The 0-th element of `@100` is mutated to 42. So, **D** is the correct answer.

(Task 17 of 20) The following program defines two variables but creates nothing on the heap.

Lispy | 

Lispy [Run ▶]

Scala 3

```
(defvar x 2)  val x = 2
(defvar y 3)  val y = 3
x             println(x)
y             println(y)
```

The following program defines no variables but creates two things on the heap.

Lispy [Run ▶]

Scala 3

```
(mvec 1 (mvec 2 3))  println((1, (2, 3)))
```

What did you learn from this pair of programs?

nothing

53

(Task 18 of 20)

54

- Creating a vector does not inherently create a binding.
- Creating a binding does not necessarily alter the heap.

Any feedback regarding these statements? Feel free to skip this question.

55

No real link to those two programs

56

Thank you!

57

(Task 19 of 20) Here is a program that confused many students

Lispy | 

Lispy [Run ▶]

Scala 3

```
(defvar v (mvec 1 2 3 4))  val v = Buffer(1, 2, 3, 4)
(defvar vv (mvec v v))     val vv = Buffer(v, v)
(vec-set! (vec-ref vv 1) 0 100)  vv(1)(0) = 100
vv                             println(vv)
```

Please

1. Run this program in the stacker by clicking one of the [Run ▶] buttons above;
2. The stacker would show how this program produces its result(s);
3. Keep clicking ▶▶ Next until you reach a configuration that you find particularly helpful;
4. Click  Share This Configuration to get a link to your configuration;
5. Submit your link below;

https://smol-tutor.xyz/stacker/?syntax=Lispy&randomSeed=smol-tutor&hole=%E2%80%A2&nNext=1&program=%0A%28defvar+v+%28mvec+1+2+3+4%29%29%0A%28defvar+vv+%28mvec+v+v%29%29%0A%28vec-set%21+%28vec-ref+vv+1%29+0+100%29%0Avv%0A&readOnlyMode=

59

(Task 20 of 20) Please write a couple of sentences to explain how your configuration explains the result(s) of the program.

60

Shows that there is only one v

61

Let's review what we have learned in this tutorial.

62

A vector can be referred to by more than one variable and even by other vectors (including itself). Referring to a vector does not create a copy of the vector; rather, they share the same vector. Specifically

- Binding a vector to a new variable does not create a copy of that vector.
- Vectors that are passed to a function in a function call do not get copied.
- Creating new vectors that refer to existing ones does not create new copies of the existing vectors.

The references share the same vector. That is, vectors can be **aliased**.

In SMoL, each vector has its own unique **heap address** (e.g., @100 and @200). The mapping from addresses to vectors is called the **heap**.

(**Note:** we use @ddd (e.g., @123, @200, and @100) to represent heap addresses. Heap addresses are *random*. The numbers don't mean anything.)

- Creating a vector does not inherently create a binding.
- Creating a binding does not necessarily alter the heap.

You have finished this tutorial 🎉🎉🎉

Please the finished tutorial to a PDF file so you can review the content in the future. **Your instructor (if any) might require you to submit the PDF.**

Start time: 1780301701966